Task III: Open Task

Quantum Machine Learning and Variational Optimization:
A Mathematical Framework

Abhinaba Ghosh

March 19, 2026

1 Introduction and Motivation

Quantum Machine Learning (QML) has grown out of the overlap between quantum information science and classical statistical learning [1]. The basic bet is straightforward: parameterized quantum circuits (PQCs) live in an exponentially large Hilbert space, $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$, and that sheer dimensionality should let them encode correlations that no polynomial-size classical model can efficiently represent. Whether this translates into genuine learning advantages on practical datasets is still debated, but the formalism itself is compelling enough to warrant serious investigation.

What I aim to do here is lay out the formal machinery behind variational quantum algorithms—both for supervised learning and for combinatorial optimization. I will start with the workhorse methods, VQE [2] and QAOA [3], then cover some more recent proposals like quantum GANs [4] and quantum graph neural networks [12]. Along the way, we cannot avoid confronting the trainability issues that barren plateaus introduce [14]; they are arguably the single biggest obstacle standing between current theory and any real-world deployment.

2 The Variational Framework

2.1 Parameterized Quantum Circuits as Unitary Maps

Definition 2.1 (Parameterized Quantum Circuit)**.** *A PQC is a unitary operator $U(\boldsymbol{\theta}) : \mathcal{H} \to \mathcal{H}$ admitting a decomposition*

$$U(\boldsymbol{\theta}) = \prod_{\ell=1}^{L} \left[W_\ell \, e^{-i\,\theta_\ell\, G_\ell/2} \right], \tag{1}$$

where $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_L) \in \mathbb{R}^L$, each $G_\ell \in \{X, Y, Z\}^{\otimes n}$ is a tensor–product Pauli generator with $G_\ell^2 = \mathbb{1}$, and W_ℓ are fixed (parameter-free) entangling unitaries composed of CNOT or CZ gates [5].

Explicitly, the single-qubit rotation gates in Eq. (1) take the form

$$\mathrm{R}_X(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \quad \mathrm{R}_Y(\theta) = e^{-i\theta Y/2}, \quad \mathrm{R}_Z(\theta) = e^{-i\theta Z/2}. \tag{2}$$

2.2 Cost Function and Expectation Values

Given a Hermitian observable $\hat{O} = \hat{O}^\dagger$ and an initial state $|\psi_0\rangle$ (typically $|0\rangle^{\otimes n}$), the cost function is

$$C(\boldsymbol{\theta}) \;=\; \langle\psi_0|\, U^\dagger(\boldsymbol{\theta})\, \hat{O}\, U(\boldsymbol{\theta})\, |\psi_0\rangle \;=\; \mathrm{Tr}\big[\hat{O}\;\rho(\boldsymbol{\theta})\big], \tag{3}$$

where the parameterized state and its density matrix are

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})\, |\psi_0\rangle\,, \qquad \rho(\boldsymbol{\theta}) = |\psi(\boldsymbol{\theta})\rangle\langle\psi(\boldsymbol{\theta})|\,. \tag{4}$$

2.3 Parameter-Shift Rule for Analytic Gradients

Proposition 2.1 (Parameter-Shift Rule, Mitarai et al. [6]). *If $G_\ell^2 = \mathbb{1}$, the partial derivative of the cost with respect to θ_ℓ is exactly*

$$\frac{\partial C}{\partial \theta_\ell} \;=\; \frac{1}{2}\Big[C\big(\boldsymbol{\theta} + \tfrac{\pi}{2}\,\mathbf{e}_\ell\big) - C\big(\boldsymbol{\theta} - \tfrac{\pi}{2}\,\mathbf{e}_\ell\big)\Big], \tag{5}$$

where $\mathbf{e}_\ell$ is the ℓ-th standard basis vector in $\mathbb{R}^L$.

Why does this matter so much in practice? Numerical finite differences are notoriously fragile when your function evaluations come from quantum hardware—shot noise contaminates every estimate. The parameter-shift rule sidesteps that entirely by giving closed-form gradients from just two circuit runs per parameter. So the full gradient $\nabla_{\boldsymbol{\theta}} C$ costs $2L$ evaluations, no approximation involved [7]. For anyone who has tried to optimize a noisy black-box function, that kind of guarantee is hard to overstate.

3 Quantum Machine Learning: Supervised Classification

3.1 Feature Encoding via Quantum Embedding Maps

Any QML pipeline has to start with encoding: we need a way to inject a classical data point $\mathbf{x} \in \mathbb{R}^d$ into the Hilbert space through some unitary $S(\mathbf{x})$. The two standard approaches each come with their own trade-offs.

Amplitude Encoding. Normalize $\mathbf{x} \mapsto \tilde{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|$ and encode

$$S_{\mathrm{amp}}(\mathbf{x})\, |0\rangle^{\otimes n} = \sum_{j=0}^{2^n-1} \tilde{x}_j\, |j\rangle\,, \qquad n = \lceil \log_2 d \rceil. \tag{6}$$

On paper this is fantastically efficient—2^n features crammed into n qubits. In reality, the state preparation circuits required to do this are deep enough that coherence times on current devices become a serious bottleneck.

Angle (Rotation) Encoding. Map each feature to a rotation angle on a dedicated qubit:

$$S_{\mathrm{ang}}(\mathbf{x}) = \bigotimes_{j=1}^{d} \mathrm{R}_Y(x_j) = \bigotimes_{j=1}^{d} e^{-i\,x_j\,Y_j/2}. \tag{7}$$

3.2 The Quantum Kernel Method

Definition 3.1 (Quantum Kernel, Havlíček et al. [8]). *The quantum kernel induced by the encoding map S is*

$$\kappa(\mathbf{x}, \mathbf{x}') \;=\; \left|\langle 0|^{\otimes n}\, S^\dagger(\mathbf{x})\, S(\mathbf{x}')\, |0\rangle^{\otimes n}\right|^2 \;=\; \left|\mathrm{Tr}\big[\rho(\mathbf{x})\, \rho(\mathbf{x}')\big]\right|. \tag{8}$$

The idea is deceptively simple. Rather than hand-engineering feature maps in $\mathbb{R}^D$ the way classical kernel methods do, we push the data into quantum state space and let the Hilbert space geometry define the similarity measure for us. Mercer's theorem guarantees that κ induces a valid reproducing kernel Hilbert space (RKHS), so the Gram matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ slots directly into a standard SVM solver on the classical side. I find this particularly elegant because it cleanly separates the "quantum part" (estimating overlaps on hardware) from the "classical part" (solving the convex optimization).

4 Combinatorial Optimization via QAOA

4.1 Problem Formulation

A large class of NP-hard combinatorial problems—MaxCut, scheduling, certain satisfiability variants—reduce to minimizing a cost function over n binary variables $z \in \{0,1\}^n$:

$$C_{\mathrm{cl}}(z) = \sum_{\langle i,j\rangle \in E} w_{ij}\, z_i\, (1 - z_j). \tag{9}$$

The standard trick to make this quantum-compatible is the substitution $z_i \mapsto (\mathbb{1} - Z_i)/2$, which lifts the cost to an operator on $\mathcal{H}$:

$$\hat{C} = \sum_{\langle i,j\rangle} \frac{w_{ij}}{2}\big(\mathbb{1} - Z_i Z_j\big). \tag{10}$$

4.2 The QAOA Ansatz

Definition 4.1 (QAOA$_p$, Farhi et al. [3]). *For depth $p \geq 1$, the QAOA state with parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta}) \in \mathbb{R}^{2p}$ is*

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{k=1}^{p} e^{-i\beta_k \hat{B}}\, e^{-i\gamma_k \hat{C}}\; |+\rangle^{\otimes n}\,, \tag{11}$$

where $\hat{B} = \sum_{i=1}^n X_i$ is the mixer Hamiltonian and $|+\rangle^{\otimes n} = H^{\otimes n}\, |0\rangle^{\otimes n}$.

What is actually going on here, physically? The alternating structure is really a Trotterized version of adiabatic state preparation. Each $e^{-i\gamma_k \hat{C}}$ layer stamps phase kicks onto computational basis states proportional to their cost—so "bad" bitstrings pick up large phases. Then the mixer $e^{-i\beta_k \hat{B}}$ flips bits, redistributing amplitude away from those penalized states. With enough rounds and well-chosen angles, the amplitude concentrates on low-cost configurations. Farhi et al. showed that even at $p = 1$ on unweighted MaxCut, the expected cut value is at least ≈ 0.6768 of the optimum [3]. As p grows, the approximation ratio improves, and in the $p \to \infty$ limit the adiabatic theorem kicks in to guarantee convergence to the exact optimum.

5 Combinatorial Optimization through Quantum Annealing (QA)

Quantum Annealing is not a gate-model algorithm, but it is the conceptual ancestor of QAOA and worth discussing on its own terms [13]. The setup is a time-dependent Hamiltonian that interpolates between the transverse-field mixer $\hat{B} = \sum_i X_i$ and the problem Hamiltonian $\hat{C}$:

$$\hat{H}(t) = \left(1 - \frac{t}{T}\right)\hat{B} + \frac{t}{T}\hat{C}. \tag{12}$$

At $t = 0$ the system sits in the easily prepared ground state of $\hat{B}$. As t increases toward T, the Hamiltonian morphs into $\hat{C}$, and—provided the sweep is slow enough relative to the minimum spectral gap—the adiabatic theorem guarantees the system tracks the instantaneous ground state all the way to the optimum. The intuition from classical simulated annealing is helpful here: classical SA must thermally hop over energy barriers, while QA can tunnel through them via quantum fluctuations from the transverse field. Whether this tunneling advantage survives in the presence of realistic noise and finite temperature is an active and contentious area of research.

6 Variational Quantum Eigensolver (VQE)

Given a molecular Hamiltonian [2]:

$$\hat{H}_{\text{mol}} = \sum_{\alpha} c_\alpha \, \hat{P}_\alpha, \qquad \hat{P}_\alpha \in \{I, X, Y, Z\}^{\otimes n}, \tag{13}$$

VQE seeks the ground-state energy $E_0 = \min_{\boldsymbol{\theta}} \langle\psi(\boldsymbol{\theta})| \hat{H}_{\text{mol}} |\psi(\boldsymbol{\theta})\rangle$.

The logic is almost embarrassingly simple once you see it: the Rayleigh–Ritz variational principle tells us that $\langle\psi(\boldsymbol{\theta})| \hat{H}_{\text{mol}} |\psi(\boldsymbol{\theta})\rangle \geq E_0$ for every $\boldsymbol{\theta}$, so we just need to keep pushing the energy estimate downward. The nontrivial part—and where most of the engineering effort goes—is the measurement strategy. You cannot measure $\hat{H}_{\text{mol}}$ directly as a single observable; instead, you partition its Pauli terms into groups of mutually commuting operators and measure each group separately. Clever grouping (e.g., via graph coloring on the commutativity graph) is crucial for keeping the total shot budget manageable on real hardware.

7 Advanced Variational Algorithms

Below I sketch several extensions that push it into deep learning, generative modeling, linear algebra, graph-structured data, and analog computation.

7.1 Quantum Convolutional Neural Networks (QCNN)

QCNNs borrow the hierarchical coarse-graining idea from classical CNNs: alternating convolutional unitaries U_c on neighboring qubits with pooling steps U_p that trace out a subset of the register [9].

$$\rho_{k+1} = \mathrm{Tr}_{\text{pool}}\big[U_c(\boldsymbol{\theta}_k)\, \rho_k\, U_c^\dagger(\boldsymbol{\theta}_k)\big]. \tag{14}$$

What makes QCNNs more than just a superficial analogy is the connection to Wilson's renormalization group in condensed matter. Each pooling layer integrates out short-range degrees of freedom, progressively exposing long-range correlations. Pesah et al. proved that this structure is inherently resistant to barren plateaus [10], which makes QCNNs one of the few architectures with a provable trainability guarantee. In my view, this RG connection is one of the most intellectually satisfying results in recent QML literature.

7.2 Quantum Generative Adversarial Networks (qGANs)

A qGAN pits a parameterized quantum generator $G(\boldsymbol{\theta})$ against a discriminator $D(\boldsymbol{\phi})$ in the usual minimax game [4]:

$$\min_{\boldsymbol{\theta}} \max_{\boldsymbol{\phi}} V(G_{\boldsymbol{\theta}}, D_{\boldsymbol{\phi}}) = \mathbb{E}_{x\sim P_{\text{data}}}[\log D_{\boldsymbol{\phi}}(x)] + \mathbb{E}_z[\log(1 - D_{\boldsymbol{\phi}}(G_{\boldsymbol{\theta}}(z)))]. \tag{15}$$

The key difference from the classical version is that the generator's output is not a vector of floats—it is a genuine quantum probability distribution, a Born-rule distribution over measurement outcomes. This is interesting for a very specific reason: loading an arbitrary classical distribution into a quantum state (so-called "state preparation") is generically expensive, often requiring $O(2^n)$ gates. If a qGAN can learn to approximate the target distribution variationally, it effectively solves the state-preparation bottleneck that plagues quantum Monte Carlo, option pricing, and similar algorithms.

7.3 Variational Quantum Linear Solver (VQLS)

The HHL algorithm promises exponential speedup for solving $A\mathbf{x} = \mathbf{b}$, but its circuit depth puts it well beyond NISQ reach. VQLS is the pragmatic compromise [11]. We decompose $A = \sum_j c_j A_j$ into a sum of implementable unitaries, prepare a trial state $|x(\boldsymbol{\theta})\rangle$, and minimize

$$C_L(\boldsymbol{\theta}) = 1 - \frac{|\langle b| A |x(\boldsymbol{\theta})\rangle|^2}{\langle x(\boldsymbol{\theta})| A^\dagger A |x(\boldsymbol{\theta})\rangle}. \tag{16}$$

This cost function has a clean geometric interpretation: $C_L = 0$ exactly when $A|x(\boldsymbol{\theta})\rangle$ is proportional to $|b\rangle$, i.e., when the normalized solution direction has been found. The optimizer does not need to invert any matrix; it just chases a fidelity-like overlap toward unity.

7.4 Quantum Graph Neural Networks (QGNN)

Many problems in chemistry, logistics, and network science are naturally graph-structured, and QGNNs are designed to exploit that structure directly [12]. The ansatz applies layers of Hamiltonian evolutions tied to the graph's edge set:

$$\hat{U}_{\text{QGNN}}(\boldsymbol{\eta}, \boldsymbol{\theta}) = \prod_{p=1}^{P} \left[\prod_{q=1}^{Q} e^{-i\eta_{pq}\hat{H}_q(\boldsymbol{\theta})} \right]. \tag{17}$$

A particularly neat trick is parameter tying: if two edges are structurally equivalent under the graph automorphism group, their parameters are forced to be identical. This is the quantum analogue of weight sharing in classical CNNs. The payoff is twofold—fewer free parameters to optimize, and the circuit automatically respects the problem's symmetry, which tends to smooth out the loss landscape and improve convergence in practice.

8 Trainability: The Barren Plateau Problem

Proposition 8.1 (Barren Plateau, McClean et al. [14]). *For a PQC forming a unitary 2-design over* $\mathrm{SU}(2^n)$*, the variance of the cost gradient vanishes exponentially:*

$$\mathrm{Var}_{\boldsymbol{\theta}}\left[\frac{\partial C}{\partial \theta_\ell}\right] \leq \frac{2\,\|\hat{O}\|_\infty^2}{2^{2n}}, \tag{18}$$

where $\|\hat{O}\|_\infty$ *is the operator norm of the cost observable.*

This is, frankly, a devastating result if taken at face value: it says that for sufficiently expressive random circuits, the gradient signal drowns in shot noise exponentially fast with qubit count. Known workarounds include restricting to local cost functions, training layer-by-layer rather than all-at-once, and replacing vanilla gradient descent with the Quantum Natural Gradient (QNG) [15]. QNG uses the Fubini–Study metric on the state manifold to precondition the gradient, which amounts to a Riemannian optimization step that accounts for the non-Euclidean geometry of quantum state space.

9 Conclusion

We are squarely in the NISQ era—lots of theoretical promise, severe hardware constraints. Qubit counts are limited, coherence windows are short, and gate fidelities leave much to be desired. Against that backdrop, variational algorithms are really the only viable game in town for extracting useful computation from near-term devices. Their hybrid structure—classical optimizer in the outer loop, quantum circuit in the inner loop—means they can tolerate a certain amount of noise without catastrophic failure, which is exactly what you need when your hardware is imperfect.

Looking back at what we have covered—the parameter-shift rule, quantum kernels, QAOA, VQE, QCNNs, qGANs, VQLS, QGNNs—I think the field has clearly moved past the "let's just try random circuits and see what happens" phase. The formalism is now grounded in linear algebra, Lie theory, and statistical learning in a way that makes it possible to prove things rather than just observe them empirically.

That said, the elephant in the room remains trainability. The barren plateau results [14] are a hard constraint: one cannot simply throw more parameters at the problem and hope for the best. The generic hardware-efficient ansatz that was popular three or four years ago is, in hindsight, a dead end for anything beyond a handful of qubits.

Where I see the most productive path forward is in what people are calling *geometric* or *equivariant* QML. The core idea is to bake the symmetries of your problem—permutation invariance, translational symmetry, whatever the data naturally possesses—directly into the circuit architecture by restricting $U(\boldsymbol{\theta})$ to the commutant of the symmetry group's representation. This is not just an aesthetic preference; it provably shrinks the parameter space and, in many cases, provably avoids barren plateaus. The analogy to classical deep learning is apt: CNNs did not beat fully-connected networks because they were more expressive—they won because translational equivariance was the right inductive bias for images. The same logic should apply here.

If QML is going to matter outside of academic benchmarks, we need to stop treating quantum circuits as generic function approximators and start designing them around the mathematical structure of the problems we actually care about.

References

[1] J. Biamonte *et al.*, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[2] A. Peruzzo *et al.*, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, no. 1, p. 4213, 2014.

[3] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[4] C. Zoufal, A. Lucchi, and S. Woerner, "Quantum generative adversarial networks for learning and loading random distributions," *npj Quantum Information*, vol. 5, no. 1, p. 103, 2019.

[5] M. Benedetti *et al.*, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, no. 4, p. 043001, 2019.

[6] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, "Quantum circuit learning," *Physical Review A*, vol. 98, no. 3, p. 032309, 2018.

[7] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, "Evaluating analytic gradients on quantum hardware," *Physical Review A*, vol. 99, no. 3, p. 032331, 2019.

[8] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[9] I. Cong, S. Choi, and M. D. Lukin, "Quantum convolutional neural networks," *Nature Physics*, vol. 15, no. 12, pp. 1273–1278, 2019.

[10] A. Pesah, M. Cerezo, S. Wang, T. Volkoff, A. T. Sornborger, and P. J. Coles, "Absence of barren plateaus in quantum convolutional neural networks," *Physical Review X*, vol. 11, no. 4, p. 041011, 2021.

[11] C. Bravo-Prieto *et al.*, "Variational quantum linear solver," *arXiv preprint arXiv:1909.05820*, 2019.

[12] G. Verdon *et al.*, "Quantum Graph Neural Networks," *arXiv preprint arXiv:1909.12264*, 2019.

[13] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Physical Review E*, vol. 58, no. 5, p. 5355, 1998.

[14] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, no. 1, p. 4812, 2018.

[15] J. Stokes, J. Izaac, N. Killoran, and G. Carleo, "Quantum Natural Gradient," *Quantum*, vol. 4, p. 269, 2020.